

DATA SCIENCE CAPSTONE PROJECT

MANUFACTURING EQUIPMENT OUTPUT PREDICTION USING REGRESSION

Project Report

Submitted by:

Name: Mahammad Shahid Jamadar (T170800223)

Yash Kapse (T170800213)

Adarsh Shelke (T170800211)

Tanaya Mote (T170800221)

Yashashree Mene (T170800251)

Shejal Thakur (T170800205)

Batch: AI/ML (02)

College: JSPM's Jayawantrao Sawant College of Engineering, Pune

Academic Year: 2026–2027

Abstract:

Manufacturing industries continuously aim to improve production efficiency, reduce downtime, and optimize machine performance. In injection molding, the number of parts produced per hour depends on several machine operating parameters, material properties, operator-related factors, and time-based conditions. Accurate prediction of machine output can therefore support production planning and help identify machines or operating conditions that require improvement.

This project focuses on developing a machine-learning model to predict the hourly production output, represented by **Parts_Per_Hour**, of injection-molding equipment. The dataset consists of 1,000 records containing parameters such as injection temperature, injection pressure, cycle time, cooling time, material viscosity, machine age, operator experience, machine utilization, and other categorical and time-based features. The data was preprocessed by handling missing values, encoding categorical variables, scaling numerical features, and performing feature engineering.

Multiple regression approaches were evaluated, including Linear Regression, Random Forest, Polynomial Regression of Degree 2, and Polynomial Regression of Degree 3. Among the tested models, **Polynomial Regression Degree 3** achieved the best performance with an MAE of **2.2991 Parts/Hour**, RMSE of **2.9440 Parts/Hour**, and test R^2 of **0.9336**. Five-fold cross-validation produced a mean R^2 of **0.9177**, indicating reasonably consistent performance across different data subsets. The final model was also integrated into a FastAPI and Docker-based deployment workflow.

1. Introduction:

Manufacturing industries use automated machines to produce large quantities of products with consistent quality and efficiency. Injection molding is one such manufacturing process used for producing plastic components. The productivity of an injection-molding machine can be influenced by several factors, including injection temperature, injection pressure, cycle time, cooling time, material properties, machine age, operator experience, and machine utilization.

The manufacturing company considered in this project wants to improve production efficiency by predicting the number of parts that can be produced by a machine in one hour. The official project objective is to use machine operating parameters and material properties to predict hourly machine output. Such predictions can help production teams optimize machine settings, plan production schedules, and identify underperforming machines.

Machine-learning regression techniques can be used to learn relationships between input parameters and production output. In this project, the target variable is **Parts_Per_Hour**, which represents the hourly production output of the machine.

The project follows a supervised-learning approach because the model is trained using historical data in which the expected production output is already available.

2. Problem Statement:

The objective of this project is to develop a machine-learning model that can predict the hourly output of an injection-molding machine using machine operating parameters, material properties, operator information, and time-based features.

The predicted output can be used to:

- Optimize machine operating conditions.
- Improve production efficiency.
- Support production scheduling.
- Identify machines with lower-than-expected output.
- Assist production teams in making data-driven decisions.

The official project defines the task as a supervised-learning regression problem with **Parts_Per_Hour** as the target variable.

3. Objectives:

The major objectives of this project are:

1. To analyze manufacturing equipment data and understand the factors affecting production output.
 2. To preprocess the dataset by handling missing values and categorical variables.
 3. To perform exploratory data analysis to identify important relationships between machine parameters and production output.
 4. To perform feature engineering using relevant machine and time-based variables.
 5. To develop regression models for predicting **Parts_Per_Hour**.
 6. To compare the performance of different regression approaches.
 7. To select the best-performing model using appropriate evaluation metrics.
 8. To analyze important feature relationships.
 9. To integrate the final model into a deployment pipeline using FastAPI and Docker.
-

4. Dataset Description:

The dataset used for this project is **manufacturing_dataset_1000_samples**. It contains **1,000 records** and uses **Parts_Per_Hour** as the target variable. The official dataset specification contains 17 primary input features and the hourly production output as the target.

The target variable has the following characteristics:

Parameters	Value
Number of records	1,000
Target variable	Parts_Per_Hour
Minimum target value	5.0
Maximum target value	68.6

Mean target value	29.30 Parts/Hour
-------------------	------------------

These values are from the actual model-performance report.

Major input features:

The dataset contains machine, material, operator, and time-related variables such as:

- Injection Temperature
- Injection Pressure
- Cycle Time
- Cooling Time
- Material Viscosity
- Ambient Temperature
- Machine Age
- Operator Experience
- Maintenance Hours
- Shift
- Machine Type
- Material Grade
- Day of Week
- Temperature Pressure Ratio
- Efficiency Score
- Machine Utilization
- Hour
- Day
- Month
- Day of Week

The final model report lists these features used in the modeling process.

```
import pandas as pd
import numpy as np
import joblib
```

```
df = pd.read_csv('manufacturing_dataset_1000_samples.csv')
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 19 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Timestamp                            1000 non-null   object
1   Injection_Temperature                1000 non-null   float64
2   Injection_Pressure                  1000 non-null   float64
3   Cycle_Time                          1000 non-null   float64
4   Cooling_Time                        1000 non-null   float64
5   Material_Viscosity                  980 non-null    float64
6   Ambient_Temperature                 980 non-null    float64
7   Machine_Age                         1000 non-null   float64
8   Operator_Experience                 980 non-null    float64
9   Maintenance_Hours                  1000 non-null   int64
10  Shift                               1000 non-null   object
11  Machine_Type                        1000 non-null   object
12  Material_Grade                      1000 non-null   object
13  Day_of_Week                         1000 non-null   object
14  Temperature_Pressure_Ratio          1000 non-null   float64
15  Total_Cycle_Time                    1000 non-null   float64
16  Efficiency_Score                    1000 non-null   float64
17  Machine_Utilization                 1000 non-null   float64
18  Parts_Per_Hour                      1000 non-null   float64
dtypes: float64(13), int64(1), object(5)
memory usage: 148.6+ KB
```

Figure 1: Loading and initial inspection of the manufacturing dataset

5. Data Preprocessing:

Data preprocessing was performed before training the machine-learning models. The purpose of preprocessing was to improve data quality and convert the raw dataset into a form suitable for machine learning.

5.1 Handling Missing Values

The project dataset contains missing values in:

- `Material_Viscosity`
- `Ambient_Temperature`
- `Operator_Experience`

The official project specification states that each of these columns contains 20 missing values. Numerical missing values can be handled using mean or median imputation.

Median imputation was used for the relevant numerical variables.

A screenshot of a Jupyter Notebook interface. At the top, the code `df.isnull().sum()` is entered. Below it, a table displays the count of missing values for each column in the dataset. The columns are listed on the left, and the corresponding counts are on the right.

Timestamp	0
Injection_Temperature	0
Injection_Pressure	0
Cycle_Time	0
Cooling_Time	0
Material_Viscosity	20
Ambient_Temperature	20
Machine_Age	0
Operator_Experience	20
Maintenance_Hours	0
Shift	0
Machine_Type	0
Material_Grade	0
Day_of_Week	0
Temperature_Pressure_Ratio	0
Total_Cycle_Time	0

```
df['Material_Viscosity'] = df['Material_Viscosity'].fillna(
    df['Material_Viscosity'].median()
)

df['Ambient_Temperature'] = df['Ambient_Temperature'].fillna(
    df['Ambient_Temperature'].median()
)

df['Operator_Experience'] = df['Operator_Experience'].fillna(
    df['Operator_Experience'].median()
)
```

Figure 2: Identification and treatment of missing values

5.2 Categorical Variable Encoding:

The dataset contains categorical variables such as:

- Shift
- Machine Type
- Material Grade

- Day of Week

These variables cannot be directly used by most machine-learning algorithms in their text form. Therefore, categorical variables were converted into numerical representations using One-Hot Encoding. The project specification recommends One-Hot Encoding or Label Encoding for these variables.

One-Hot Encoding creates separate binary columns representing the different categories.

5.3 Feature Scaling:

Numerical variables can have different ranges. For example, machine age and injection temperature have very different numerical scales.

Therefore, numerical features were standardized using **StandardScaler**. The official project recommends StandardScaler or MinMaxScaler for numerical features.

```
# =====  
# 1. IMPORT LIBRARIES  
# =====  
  
import pandas as pd  
import numpy as np  
import joblib  
  
from sklearn.model_selection import train_test_split, KFold  
from sklearn.preprocessing import PolynomialFeatures, StandardScaler  
from sklearn.compose import ColumnTransformer  
from sklearn.pipeline import Pipeline  
from sklearn.impute import SimpleImputer  
from sklearn.preprocessing import OneHotEncoder  
from sklearn.linear_model import LinearRegression  
from sklearn.metrics import (  
    mean_absolute_error,  
    mean_squared_error,  
    r2_score  
)
```

Figure 3: Categorical encoding and feature scaling

6. Feature Engineering:

Feature engineering was performed to create additional useful information from existing variables.

Temperature Pressure Ratio

The feature was calculated as:

$$\text{Temperature_Pressure_Ratio} = \text{Injection_Temperature} / \text{Injection_Pressure}$$

This feature represents the relationship between the injection temperature and pressure.

Another feature, **Total_Cycle_Time**, was initially created using:

$$\text{Total_Cycle_Time} = \text{Cycle_Time} + \text{Cooling_Time}$$

However, this feature was excluded from the final polynomial model because it is directly derived from the two existing variables and therefore does not provide independent information.

Time-related information was also represented using features such as:

- Hour
- Day
- Month
- DayOfWeek

These features allow the model to capture possible time-based patterns.

```
numeric_features = [  
    "Injection_Temperature",  
    "Injection_Pressure",  
    "Cycle_Time",  
    "Cooling_Time",  
    "Material_Viscosity",  
    "Ambient_Temperature",  
    "Machine_Age",  
    "Operator_Experience",  
    "Maintenance_Hours",  
    "Temperature_Pressure_Ratio",  
    "Efficiency_Score",  
    "Machine_Utilization",  
    "Hour",  
    "Day",  
    "Month",  
    "DayOfWeek"  
]
```

```
categorical_features = [  
    "Shift",  
    "Machine_Type",  
    "Material_Grade",  
    "Day_of_Week"  
]  
  
# Polynomial features only for the most important variables  
polynomial_features = [  
    "Cycle_Time",  
    "Injection_Temperature",  
    "Operator_Experience"  
]
```

Figure 4: Definition and selection of numerical, categorical and polynomial features

7. Exploratory Data Analysis:

Exploratory Data Analysis (EDA) was performed to understand the distribution of the target variable and the relationships between machine parameters and production output.

7.1 Target Variable Distribution

The target variable **Parts_Per_Hour** ranges from **5.0 to 68.6 Parts/Hour**, with an average value of **29.30 Parts/Hour**.

7.2 Feature Relationships

Correlation analysis was performed to identify relationships between important numerical features and **Parts_Per_Hour**.

The strongest reported relationship was between **Cycle_Time** and **Parts_Per_Hour**, with a correlation of **-0.819**.

This strong negative relationship indicates that longer machine cycle times are generally associated with lower hourly production output.

Other important relationships were:

Feature	Correlation with Parts_Per_Hour
Cycle_Time	-0.819
Total_Cycle_Time	-0.793
Operator_Experience	0.297
Efficiency_Score	0.287
Injection_Temperature	0.184
Machine_Utilization	0.175
Machine_Age	-0.175

8. Model Development:

The target variable was defined as: **Parts_Per_Hour**

All other relevant processed variables were used as input features. The project specification defines **Parts_Per_Hour** as the target and the remaining processed variables as features.

The dataset was divided into training and testing subsets using an **80:20 split**.

8.1 Models Evaluated

Four regression approaches were evaluated:

1. Linear Regression
2. Random Forest
3. Polynomial Regression Degree 2
4. Polynomial Regression Degree 3

The purpose of comparing multiple models was to determine which approach could provide the most accurate predictions.

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

final_model.fit(
    X_train,
    y_train
)

print("Training completed successfully!")

# =====
# 11. TEST PREDICTIONS
# =====

y_pred = final_model.predict(X_test)

Training Degree-3 Polynomial Regression...
Training completed successfully!
```

Figure 7: Training and testing data split and model training

9. Model Comparison:

The models were compared using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and R^2 score.

Model	MAE	RMSE	R^2
Linear Regression	2.7063	3.4915	0.9066
Random Forest	3.4648	4.4060	0.8512
Polynomial Regression Degree 2	2.4843	3.1924	0.9219
Polynomial Regression Degree 3	2.2991	2.9440	0.9336

The results show that Polynomial Regression Degree 3 achieved the lowest MAE and RMSE and the highest R^2 score among the evaluated models.

Therefore, **Polynomial Regression Degree 3** was selected as the final model.

10. Final Model Evaluation:

The final Polynomial Regression Degree 3 model achieved the following performance:

Metric	Result
MAE	2.2991 Parts/Hour
RMSE	2.9440 Parts/Hour
Test R^2	0.9336
Test R^2 Percentage	93.36%

The R^2 value of 0.9336 means that the model explains approximately **93.36% of the variance in machine output on the held-out test data**. It should not be interpreted as classification accuracy.

```

mae = mean_absolute_error(
    y_test,
    y_pred
)

mse = mean_squared_error(
    y_test,
    y_pred
)

rmse = np.sqrt(mse)

r2 = r2_score(
    y_test,
    y_pred
)

print("\n")
print("=====")
print("FINAL DEGREE-3 POLYNOMIAL MODEL")
print("=====")

```

```

print("MAE :", round(mae, 4))
print("MSE :", round(mse, 4))
print("RMSE :", round(rmse, 4))
print("R² :", round(r2, 4))

print("=====")

=====
FINAL DEGREE-3 POLYNOMIAL MODEL
=====
MAE : 2.2991
MSE : 8.6673
RMSE : 2.944
R² : 0.9336
=====

```

10.1 Cross-Validation:

Five-fold cross-validation was also performed.

The individual fold R^2 values were:

Fold	R^2
1	0.9336
2	0.9236
3	0.9302
4	0.8660
5	0.9351

The mean five-fold CV R^2 was **0.9177**, with a standard deviation of **0.0262**.

This indicates that the model maintains reasonably strong performance across different subsets of the dataset.

```
=====
5-FOLD CROSS VALIDATION
=====
Fold 1 R2 : 0.9336
Fold 2 R2 : 0.9236
Fold 3 R2 : 0.9302
Fold 4 R2 : 0.8660
Fold 5 R2 : 0.9351
-----
Mean CV R2 : 0.9177
Std CV R2 : 0.0262
=====
```

11.Results:

The analysis demonstrates that machine operating conditions have a strong relationship with hourly production output.

The most significant reported relationship was between cycle time and machine output. The correlation of **-0.819** indicates that increasing cycle time is generally associated with a decrease in hourly production.

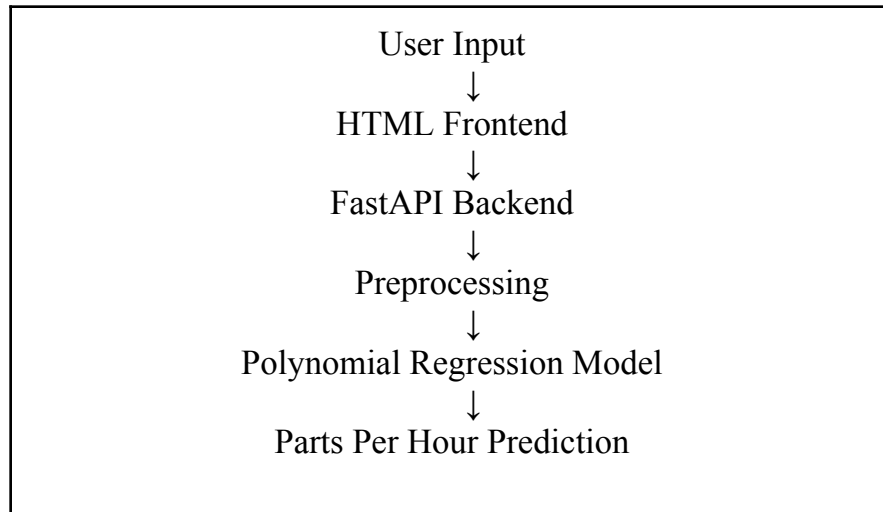
Operator experience and efficiency score showed positive relationships with production output, with correlations of 0.297 and 0.287 respectively. This suggests that more experienced operators and higher efficiency levels may be associated with improved production performance.

The final Polynomial Regression Degree 3 model achieved an R^2 of 0.9336, which was higher than the other tested models. The model also reduced MAE and RMSE compared with the original Linear Regression model.

12. Deployment:

The trained machine-learning model was integrated into a deployment pipeline using **FastAPI, an HTML frontend, and Docker**.

The deployment workflow is:



The model was containerized using Docker. The Docker image contains the FastAPI application, frontend, Python dependencies, and trained model. The application was tested locally through port 8000.

A successful example prediction from the deployed application was **33.63 Parts/Hour**.

13. Conclusion:

This project developed a machine-learning solution for predicting the hourly output of injection-molding equipment. The dataset contained 1,000 records and included machine operating parameters, material properties, operator information, and time-based features.

Data preprocessing included missing-value treatment, categorical-variable encoding, feature scaling, and feature engineering. Exploratory analysis showed that cycle time had the strongest reported relationship with hourly production output, with a correlation of -0.819 .

Multiple regression models were evaluated. Polynomial Regression Degree 3 achieved the best performance, with an MAE of **2.2991 Parts/Hour**, RMSE of **2.9440 Parts/Hour**, and R^2 of **0.9336**. Five-fold cross-validation resulted in a mean R^2 of **0.9177**, providing evidence of reasonably consistent model performance.

The final model was also integrated into a FastAPI and Docker-based deployment workflow. This demonstrates how the machine-learning model can move beyond experimentation and be used as a practical prediction tool.

Overall, the project demonstrates that regression-based machine learning can provide useful predictions of manufacturing equipment output and support data-driven production decisions.
